

tpgl2

Turki Farid

November 2025

1 Partie (c) : Implémentation de l'Horloge

L'objectif de cette partie était de modifier la classe `Horloge` pour qu'elle agisse en tant qu'observateur du `TimerService`. Elle doit s'abonner au service et afficher l'heure à chaque seconde.

1.1 Modification de la classe Horloge

Pour répondre aux exigences (c-1, c-2, c-3), la classe `Horloge` a été modifiée comme suit :

1. Elle implémente désormais l'interface `TimerChangeListener`.
2. Le constructeur a été modifié pour recevoir une instance de `TimerService` (Injection de Dépendance), garantissant ainsi la dépendance à l'abstraction et non à l'implémentation.
3. Lors de sa construction, l'horloge s'abonne au service en appelant `timerService.addTimeChangeListener`.
4. La méthode `propertyChange` (requis par l'interface) a été implémentée. Elle filtre les événements pour ne réagir qu'au changement de seconde (`SECONDE_PROP`) en appelant la méthode `afficherHeure()`.

```
[language=Java, caption=Code source de la classe Horloge (Observateur)]
package org.emp.gl.clients;
import org.emp.gl.timer.service.TimerService; import org.emp.gl.timer.service.TimerChangeListener;
public class Horloge implements TimerChangeListener
String name; TimerService timerService;
public Horloge(String name, TimerService timerService) this.name = name;
System.out.println("Horloge " + name + " initialized!");
this.timerService = timerService;
// Abonnement au service this.timerService.addTimeChangeListener(this);
public void afficherHeure() if (timerService != null) System.out.println(
name + " affiche : " + timerService.getHeures() + ":" + timerService.getMinutes()
+ ":" + timerService.getSecondes() );
@Override public void propertyChange(String prop, Object oldValue, Ob-
ject newValue) // Réagir uniquement au "tick" de la seconde if (prop.equals(TimerChangeListener.SECONDE_PROP))
```

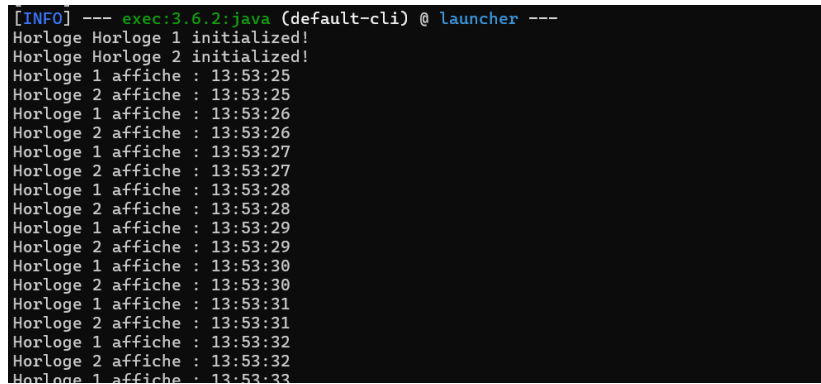
1.2 Modification du Launcher (App.java)

Conformément au point (c-4), la classe principale a été modifiée pour instancier le service (le Sujet) et deux instances de notre `Horloge` (les Observers), en injectant le service dans ces dernières.

```
[language=Java, caption=Extrait de App.java (Launcher)] private static
void testDuTimeService() // 1. Instancier le TimerService (le Sujet) TimerService timerService = new DummyTimeServiceImpl();
// 2. Instancier les Observers et injecter le sujet Horloge horloge1 = new Horloge("Horloge 1", timerService); Horloge horloge2 = new Horloge("Horloge 2", timerService);
```

1.3 Résultat de l'exécution

L'exécution du programme montre les deux horloges s'initialisant puis affichant l'heure de manière synchronisée, confirmant que le mécanisme d'Observer est fonctionnel.



```
[INFO] --- exec:3.6.2:java (default-cli) @ launcher ---
Horloge Horloge 1 initialized!
Horloge Horloge 2 initialized!
Horloge 1 affiche : 13:53:25
Horloge 2 affiche : 13:53:25
Horloge 1 affiche : 13:53:26
Horloge 2 affiche : 13:53:26
Horloge 1 affiche : 13:53:27
Horloge 2 affiche : 13:53:27
Horloge 1 affiche : 13:53:28
Horloge 2 affiche : 13:53:28
Horloge 1 affiche : 13:53:29
Horloge 2 affiche : 13:53:29
Horloge 1 affiche : 13:53:30
Horloge 2 affiche : 13:53:30
Horloge 1 affiche : 13:53:31
Horloge 2 affiche : 13:53:31
Horloge 1 affiche : 13:53:32
Horloge 2 affiche : 13:53:32
Horloge 1 affiche : 13:53:33
```

Figure 1: Résultat de l'exécution de la partie (c).

2 Partie (d) : Implémentation du Compte à Rebours

2.1 (d-1) Création et test d'un Compte à Rebours

L'objectif suivant était de créer un nouvel observateur, `CompteAREbours`, qui décrémente un entier reçu en paramètre à chaque seconde, jusqu'à zéro.

La classe a été créée sur le même modèle que `Horloge` :

1. Elle implémente `TimerChangeListener`.
2. Elle reçoit un entier (la valeur initiale) et le `TimerService` via son constructeur.
3. Elle s'abonne au service via `timerService.addTimeChangeListener(this)`.
4. Dans `propertyChange`, elle réagit à `SECONDE_PROP` en décrémentant sa valeur interne (`compteur`) tant que celle-ci est supérieure à 0.

[language=Java, caption=Code source de la classe `CompteAREbours` (partie 1)]

```
package org.emp.gl.clients;
import org.emp.gl.timer.service.TimerChangeListener; import org.emp.gl.timer.service.TimerService;
public class CompteAREbours implements TimerChangeListener
private int compteur; private TimerService timerService;
public CompteAREbours(int valeurInitiale, TimerService timerService) this.compteur
= valeurInitiale; this.timerService = timerService;
System.out.println("Nouveau Compte à rebours initialisé à " + this.compteur);
this.timerService.addTimeChangeListener(this);
@Override public void propertyChange(String prop, Object oldValue, Ob-
ject newValue)
if (prop.equals(TimerChangeListener.SECONDE_PROP))
if (this.compteur > 0) this.compteur--; System.out.println("Compte à re-
bours : " + this.compteur);
```

2.1.1 Résultat de l'exécution

La classe `App.java` a été modifiée pour instancier un `CompteAREbours` avec la valeur 5. L'exécution confirme que le compteur décrémente correctement de 5 à 0, en parallèle des horloges.

```

C:\Users\Varid\Desktop\TP1-main\TP1-main\tp-gl-master>mvn exec:java -pl launcher -Dexec.mainClass="org.esp.gl.core.launcher.App"
WARNING: A terminally deprecated method in sun.miscUnsafe has been called
WARNING: sun.misc.Unsafe::staticFieldBase has been called by com.google.inject.internal.aop.HiddenClassDefiner (File:/C:/maven-mvnd-1.0.3-windows-and64/mvn/lib/guice-5.1.0-classes.jar)
WARNING: Please consider reporting this to the maintainers of class com.google.inject.internal.aop.HiddenClassDefiner
WARNING: sun.misc.Unsafe::staticFieldBase will be removed in a future release
[INFO] Scanning for projects...
[INFO]
[INFO] -----< org.esp.gl.launcher >-----
[INFO] Building launcher 0.0.1
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- exec:3.0.2.jar (default-cli) @ launcher ---
Horloge Horloge 1 initialized!
Horloge Horloge 2 initialized!
Nouveau Compté à rebours initialisé à 5
Horloge 1 affiche : 14:14:8
Horloge 2 affiche : 14:14:8
Compté à rebours : 4
Horloge 1 affiche : 14:14:9
Horloge 2 affiche : 14:14:9
Compté à rebours : 3
Horloge 1 affiche : 14:14:10
Horloge 2 affiche : 14:14:10
Compté à rebours : 2
Horloge 1 affiche : 14:14:11
Horloge 2 affiche : 14:14:11
Compté à rebours : 1
Horloge 1 affiche : 14:14:12
Horloge 2 affiche : 14:14:12
Compté à rebours : 0
Horloge 1 affiche : 14:14:13
Horloge 2 affiche : 14:14:13
Horloge 1 affiche : 14:14:14
Horloge 2 affiche : 14:14:14
Horloge 1 affiche : 14:14:15
Horloge 2 affiche : 14:14:15
Horloge 1 affiche : 14:14:16
Horloge 2 affiche : 14:14:16
Horloge 1 affiche : 14:14:17
Horloge 2 affiche : 14:14:17
Horloge 1 affiche : 14:14:18

```

Figure 2: Résultat de l'exécution (d-1) : un compteur de 5 à 0.

2.2 (d-2) Désinscription automatique

Pour optimiser le programme, la classe `CompteAREbours` a été modifiée. Lorsque son compteur atteint 0, elle se désinscrit elle-même du `TimerService` en appelant `this.timerService.removeTimeChangeListener(this)`.

[language=Java, caption=Extrait de `CompteAREbours.java` (désinscription)]

```
@Override public void propertyChange(String prop, Object oldValue, Object  
newValue) if (prop.equals(TimerChangeListener.SECONDE_PRO))if(this.compteur > 0)this.compteur --
```

```
// AJOUT (d-2): Désinscription si le compteur atteint 0 if (this.compteur  
== 0) System.out.println("Compte à rebours terminé. Désinscription."); this.timerService.removeTimeChange
```

2.3 (d-3) & (d-4) Test de charge et analyse du bogue

L'étape (d-3) demandait d'instancier 10 `CompteAREbours` avec des valeurs aléatoires (entre 10 et 20). L'étape (d-4) nous avertissait d'un bogue potentiel.

2.3.1 Exécution

Le `App.java` a été modifié pour créer 10 compteurs en boucle. Lors de l'exécution, le programme fonctionne pendant quelques secondes, jusqu'à ce que le premier compteur atteigne 0 et tente de se désinscrire.

2.3.2 Analyse du Bogue

L'exécution échoue systématiquement en levant une `java.util.ConcurrentModificationException`, comme le montre la figure 3.

Ce bogue est un problème de concurrence classique du pattern Observer. Il se produit car :

1. Le `TimerService` (Sujet) est en train d'itérer (faire une boucle) sur sa liste d'abonnés (Observeurs) pour appeler leur méthode `propertyChange`.
2. L'un des observeurs (`CompteAREbours`) tente de modifier cette même liste (en appelant `removeTimeChangeListener`) **pendant** que le sujet est en train d'itérer dessus.
3. Les collections Java (comme `LinkedList` ou `ArrayList`) sont "fail-fast" et interdisent cette opération, provoquant l'exception.

```

Horloge 1 affiche : 14:22:43
Horloge 2 affiche : 14:22:43
Compte à rebours : 0
Compte à rebours terminé. Désinscription.
[WARNING]
java.util.ConcurrentModificationException
    at java.util.LinkedList$Itr.checkForComodification (LinkedList.java:978)
    at java.util.LinkedList$Itr.next (LinkedList.java:988)
    at org.emp.gl.time.service.impl.DummyTimeServiceImpl.secondeChanged (DummyTimeServiceImpl.java:107)
    at org.emp.gl.time.service.impl.DummyTimeServiceImpl.setSeconde (DummyTimeServiceImpl.java:102)
    at org.emp.gl.time.service.impl.DummyTimeServiceImpl.setTimeValue (DummyTimeServiceImpl.java:51)
    at org.emp.gl.time.service.impl.DummyTimeServiceImpl.timeChanged (DummyTimeServiceImpl.java:72)
    at org.emp.gl.time.service.impl.DummyTimeServiceImpl.access$000 (DummyTimeServiceImpl.java:21)
    at org.emp.gl.time.service.impl.DummyTimeServiceImpl$1.run (DummyTimeServiceImpl.java:42)
    at java.util.TimerThread.mainLoop (Timer.java:572)
    at java.util.TimerThread.run (Timer.java:522)
[INFO] -----
[INFO] BUILD FAILURE
[INFO] -----
[INFO] Total time: 11:289 s
[INFO] Finished at: 2025-11-02T14:23:43+01:00
[INFO] -----
[ERROR] Failed to execute goal org.codehaus.mojo:exec-maven-plugin:3.0.2:java (default-cli) on project launcher: An exception occurred while executing the Java class. null: ConcurrentModificationException -> [Help 1]
[ERROR]
[ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch.
[ERROR] Re-run Maven using the -X switch to enable full debug logging.
[ERROR]
[ERROR] For more information about the errors and possible solutions, please read the following articles:
[ERROR] [Help 1] http://wiki.apache.org/confluence/display/Maven/NoJreExecutionException

```

Figure 3: Résultat de l'exécution (d-4) : ConcurrentModificationException.

3 Partie (e) : Résolution du bogue

Le bogue `ConcurrentModificationException` provenait de la gestion manuelle de la liste des observateurs dans notre `DummyTimeServiceImpl`. Pour le corriger, nous avons délégué cette responsabilité à une classe utilitaire Java conçue pour cela : `java.beans.PropertyChangeSupport`.

Les modifications suivantes ont été apportées :

3.1 Modification de l'interface `TimerChangeListener`

Premièrement, notre interface d'observateur a été modifiée pour hériter de l'interface standard Java `java.beans.PropertyChangeListener`. [language=Java, caption=Interface `TimerChangeListener` modifiée] package org.emp.gl.timer.service;

```
import java.beans.PropertyChangeListener;
public interface TimerChangeListener extends PropertyChangeListener
    final static String DIXEME_DES_SECONDES_PROP = "dixième"; final static String SECONDES_PROP =
    "seconde"; final static String MINUTES_PROP = "minute"; final static String HEURES_PROP =
    "heure";
    // La méthode propertyChange(...) est maintenant héritée
```

3.2 Modification du Sujet (`DummyTimeServiceImpl`)

Le Sujet (le service) a été profondément modifié :

1. La `List<TimerChangeListener>` manuelle a été supprimée.
2. Elle a été remplacée par un objet `PropertyChangeSupport support`.
3. Cet objet est initialisé **au début** du constructeur pour éviter les `NullPointerException`.
4. Les méthodes `addTimerChangeListener` et `removeTimerChangeListener` délèguent maintenant l'appel à `support.addPropertyChangeListener(...)` et `support.removePropertyChangeListener(...)`.
5. Toutes les boucles `for` de notification ont été remplacées par un appel sécurisé : `support.firePropertyChange(...)`.

```
[language=Java, caption=Extraits de DummyTimeServiceImpl corrigé] public
class DummyTimeServiceImpl implements TimerService // ... // SUP-
PRIMÉ: List<TimerChangeListener> listeners = new LinkedList<>();
    // AJOUT: L'outil de gestion des observateurs private PropertyChangeSup-
port support;
    public DummyTimeServiceImpl() // Initialisation EN PREMIER pour éviter
NullPointerException support = new PropertyChangeSupport(this); setTimeVal-
ues(); // ... (suite du constructeur)
    @Override public void addTimerChangeListener(TimerChangeListener pl)
support.addPropertyChangeListener(pl);
```

```

@Override public void removeTimeChangeListener(TimerChangeListener pl)
support.removePropertyChangeListener(pl);
private void secondesChanged(int oldValue, int newValue) // MODIFIÉ:
Appel sécurisé qui remplace la boucle support.firePropertyChange(TimerChangeListener.SECONDE_PROP, old

```

3.3 Modification des Observeurs (Horloge et CompteARebours)

Puisque l'interface a changé, la méthode `propertyChange` de tous les observeurs a dû être mise à jour pour accepter un `PropertyChangeEvent` en paramètre, au lieu des trois paramètres (`prop`, `oldValue`, `newValue`).

```

[language=Java, caption=Nouvelle méthode propertyChange (ex: Horloge)]
import java.beans.PropertyChangeEvent;
// ... @Override public void propertyChange(PropertyChangeEvent evt) //
On récupère les informations depuis l'objet "evt" String prop = evt.getPropertyName();
if (prop.equals(TimerChangeListener.SECONDE_PROP)) afficherHeure();

```

3.4 Résultat final

Après ces modifications, l'application (avec les 10 compteurs aléatoires) a été relancée. Comme le montre la figure 4, le programme s'exécute sans lever d'exception. Les compteurs se désinscrivent correctement et les horloges continuent de fonctionner. Le bogue est résolu.


```
Compte à rebours : 2
Compte à rebours : 3
Compte à rebours : 6
Compte à rebours : 5
Compte à rebours : 1
Compte à rebours : 0
Compte à rebours terminé. Désinscription.
Compte à rebours : 1
Horloge 1 affiche : 14:39:54
Horloge 2 affiche : 14:39:54
Compte à rebours : 1
Compte à rebours : 2
Compte à rebours : 5
Compte à rebours : 4
Compte à rebours : 0
Compte à rebours terminé. Désinscription.
Compte à rebours : 0
Compte à rebours terminé. Désinscription.
Horloge 1 affiche : 14:39:55
Horloge 2 affiche : 14:39:55
Compte à rebours : 0
Compte à rebours terminé. Désinscription.
Compte à rebours : 1
Compte à rebours : 4
Compte à rebours : 3
Horloge 1 affiche : 14:39:56
Horloge 2 affiche : 14:39:56
Compte à rebours : 0
Compte à rebours terminé. Désinscription.
Compte à rebours : 3
Compte à rebours : 2
Horloge 1 affiche : 14:39:57
Horloge 2 affiche : 14:39:57
Compte à rebours : 2
Compte à rebours : 1
Horloge 1 affiche : 14:39:58
Horloge 2 affiche : 14:39:58
Compte à rebours : 1
Compte à rebours : 0
Compte à rebours terminé. Désinscription.
Horloge 1 affiche : 14:39:59
Horloge 2 affiche : 14:39:59
Compte à rebours : 0
Compte à rebours terminé. Désinscription.
Horloge 1 affiche : 14:39:00
Horloge 2 affiche : 14:39:00
```

Figure 4: Résultat de l'exécution (e) : Le bogue est corrigé.